SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science & Engineering

ASSESSMENT TOOL- 2

Course Code	**CSA4315**	**Course Title**	**Internet Programming**
CO Assessed	**CO2**	**Assessment**	**Assessment Tool 2**
Weightage	**40% of CIA**	**Total Marks**	50
CO2 Statement	DEVELOP A JAVASCRIPT-BASED FORM VALIDATION SYSTEM FOR AN ONLINE BANKING APPLICATION USING REGULAR EXPRESSIONS		
Student Name	**M Venkata Sai**	**Reg. No.**	**192425223**
Section/Batch	**Slot D / 2024**	**Date**	**05-11-2026**

Code of Conduct

I **M Venkata Sai(192425223)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: **M Venkata sai**

QUESTION 1

Develop a JavaScript Program Using DOM Manipulation and Regular Expressions to Validate Student Name, Email ID, Mobile Number and Password During Online Registration. Display Appropriate Error Messages Without Reloading the Webpage.

1.1 Aim

To develop an online student registration form using HTML, CSS and JavaScript. The form validates student name, email ID, mobile number and password using Regular Expressions. It uses DOM manipulation to display appropriate error messages without reloading the webpage.

1.2 Introduction

Online registration is an important feature of modern web applications. Students use registration forms to create accounts and provide personal information such as their name, email address, mobile number and password.

When a user enters incorrect information, the application should identify the mistake and display an appropriate error message. This process is called form validation.

JavaScript is used to make webpages interactive and dynamic. It allows developers to validate user input before submitting the form.

The Document Object Model (DOM) represents an HTML document as a collection of objects. JavaScript uses the DOM to access HTML elements, read input values, modify webpage content and display error messages.

Regular Expressions are patterns used to check whether the entered text follows a particular format.

1.3 Technologies Used

1. HTML

HTML is used to create the registration form and input fields.

2. CSS

CSS is used to provide a proper layout and appearance to the registration form.

3. JavaScript

JavaScript is used to validate the entered information and display messages.

4. DOM Manipulation

DOM manipulation is used to access HTML elements and update the webpage dynamically.

5. Regular Expressions

Regular Expressions are used to check the format of the name, email, mobile number and password.

1.4 Validation Requirements

The following validation rules are used:

Field	Validation condition
Student Name	Only letters and spaces, minimum 2 characters
Email ID	Must contain username, @ symbol and domain
Mobile Number	Exactly 10 digits, starting from 6 to 9
Password	Minimum 8 characters with uppercase, lowercase, digit and special character

1.5 Regular Expression Patterns

The following regular expressions are used to validate the input fields.

Name:

```
/^[A-Za-z ]{2,}$/
```

Email:

```
/^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$/
```

Mobile:

```
/^[6-9]\d{9}$/
```

Password:

```
/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$/
```

Explanation:

- ^ represents the beginning of the input.
- $ represents the end of the input.
- [A-Za-z] accepts uppercase and lowercase letters.
- \d accepts digits from 0 to 9.

- {2,} means at least two characters.
- {9} means exactly nine repetitions.
- (?=.*[A-Z]) checks for an uppercase letter.
- (?=.*[a-z]) checks for a lowercase letter.
- (?=.*\d) checks for a number.
- (?=.*[@$!%*?&]) checks for a special character.

1.6 System Architecture Diagram

Programming Blog & Tutorials - Unwired Learning

unwiredlearning.com

Architecture explanation:

1. The user opens the student registration webpage.
2. The user enters the required information.
3. JavaScript receives the entered values.
4. Regular Expressions check the input formats.
5. The validation result is generated.
6. Error messages are displayed for incorrect information.
7. If all inputs are valid, a success message is displayed.

1.7 Algorithm

1. Start the program.
2. Create an HTML registration form.
3. Add input fields for name, email, mobile number and password.
4. Create separate error-message areas for each field.
5. Attach a submit event listener to the form.
6. Read the entered values using DOM methods.
7. Apply Regular Expressions to validate each input.
8. If any input is incorrect, display an appropriate error message.
9. Use preventDefault() to prevent the webpage from reloading.
10. If all inputs are valid, display the registration success message.
11. Stop the program.

1.8 Program

Save the following code as registration.html.

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Registration</title>
</head>
<body>

<h2>Student Registration Form</h2>

<form id="registrationForm">

    Student Name:
    <input type="text" id="name">
    <p id="nameError"></p>

    Email ID:
    <input type="text" id="email">
    <p id="emailError"></p>

    Mobile Number:
    <input type="text" id="mobile">
    <p id="mobileError"></p>

    Password:
    <input type="password" id="password">
    <p id="passwordError"></p>
```

```
    <button type="submit">Register</button>

</form>

<h3 id="result"></h3>

<script>

document.getElementById("registrationForm")
.addEventListener("submit", function(event) {

    event.preventDefault();

    let name = document.getElementById("name").value.trim();
    let email = document.getElementById("email").value.trim();
    let mobile = document.getElementById("mobile").value.trim();
    let password = document.getElementById("password").value;

    let namePattern = /^[A-Za-z ]{2,}$/;

    let emailPattern =
    /^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$/;

    let mobilePattern = /^[6-9]\d{9}$/;

    let passwordPattern =
    /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$/;

    let valid = true;
```

```
document.getElementById("nameError").textContent = "";
document.getElementById("emailError").textContent = "";
document.getElementById("mobileError").textContent = "";
document.getElementById("passwordError").textContent = "";
document.getElementById("result").textContent = "";

if (!namePattern.test(name)) {
    document.getElementById("nameError").textContent =
    "Enter a valid student name";
    valid = false;
}

if (!emailPattern.test(email)) {
    document.getElementById("emailError").textContent =
    "Enter a valid email ID";
    valid = false;
}

if (!mobilePattern.test(mobile)) {
    document.getElementById("mobileError").textContent =
    "Enter a valid 10-digit mobile number";
    valid = false;
}

if (!passwordPattern.test(password)) {
    document.getElementById("passwordError").textContent =
    "Password must contain uppercase, lowercase, number and special character";
    valid = false;
```

```
    }

    if (valid) {
        document.getElementById("result").textContent =
        "Registration Successful!";
        document.getElementById("registrationForm").reset();
    }

});

</script>

</body>
</html>
```

1.9 Explanation of the Program

Step 1: Creating the form

HTML creates four input fields for student name, email ID, mobile number and password.

Step 2: Accessing DOM elements

The getElementById() method is used to access HTML elements using their IDs.

Step 3: Reading user input

The value property is used to retrieve the information entered by the user.

Step 4: Applying Regular Expressions

The test() method checks whether the entered information matches the required pattern.

Step 5: Displaying error messages

The textContent property displays suitable error messages when incorrect information is entered.

Step 6: Preventing webpage reload

The event.preventDefault() method prevents the normal form submission and avoids reloading the webpage.

Step 7: Displaying success message

If all fields are valid, the webpage displays the message Registration Successful.

1.10 Sample Output

Student Registration Form

Student Name: Sai Venkata

Email ID: sai@gmail.com

Mobile Number: 9876543210

Password: Sai@12345

[Register]

Registration Successful!

Invalid input example:

Student Name: Sai123

Error: Enter a valid student name.

Mobile Number: 12345

Error: Enter a valid 10-digit mobile number.

1.11 Test Cases

Test case	Input	Expected result
1	Sai Venkata	Valid name
2	Sai123	Invalid name
3	sai@gmail.com	Valid email
4	sai@gmail	Invalid email

5	9876543210	Valid mobile
6	12345	Invalid mobile
7	Sai@12345	Valid password
8	sai123	Invalid password

1.12 Advantages

1. Provides immediate feedback to users.
2. Reduces incorrect form submissions.
3. Improves the user experience.
4. Saves time during registration.
5. Uses simple JavaScript and DOM methods.
6. Prevents normal form submission until validation is completed.

1.13 Conclusion

Thus, the JavaScript student registration form successfully validates student name, email ID, mobile number and password using DOM manipulation and Regular Expressions. It displays appropriate error messages without reloading the webpage. This method improves the accuracy, reliability and usability of online registration systems.

QUESTION 2

Develop an E-Commerce Dynamic Product Search Webpage Using JavaScript DOM and AJAX to Retrieve Product Details in JSON Format from the Server and Display Matching Products Dynamically as the User Enters a Product Name.

10 Marks

2.1 Aim

To develop a dynamic product search webpage using JavaScript DOM manipulation and AJAX. The application retrieves product details from a server in JSON format and displays matching products dynamically when the user enters a product name.

2.2 Introduction

E-commerce websites contain a large number of products, such as laptops, mobile phones, electronic devices, clothing and accessories.

Customers need an easy method to search for products. A dynamic product search system allows users to enter a product name and view matching products immediately.

AJAX is used to communicate with the server in the background without reloading the complete webpage.

JavaScript is used to retrieve the search keyword, process the server response and update the webpage using DOM manipulation.

JSON is used to exchange product information between the server and the browser.

2.3 Technologies Used

HTML: Creates the search box and product display area.

CSS: Provides the appearance and layout of product information.

JavaScript: Handles user input and processes server responses.

DOM: Dynamically creates and displays product details.

AJAX: Communicates with the server without reloading the webpage.

JSON: Stores and transfers product information in a structured format.

2.4 System Architecture

타임투데브:5. 프론트엔드/백엔드 프로그래밍 환경 이해하기

timetodev.co.kr

Architecture explanation:

1. The customer enters a product name.
2. JavaScript captures the search input.
3. AJAX sends the search keyword to the server.
4. The server searches the product database.
5. Matching product details are returned in JSON format.
6. JavaScript receives and processes the JSON response.
7. DOM manipulation displays the matching products on the webpage.

2.5 Working Principle

The dynamic product search system works through the following steps.

Step 1: User input

The customer enters a product name in the search box.

Step 2: Capturing input

JavaScript uses the input event to identify changes in the search field.

Step 3: Sending AJAX request

An asynchronous request is sent to the server with the entered keyword.

Step 4: Server processing

The server searches the product database and identifies matching products.

Step 5: JSON response

The server sends the matching product details in JSON format.

Step 6: Processing JSON

JavaScript receives the response and converts it into JavaScript objects.

Step 7: Displaying results

The DOM creates product elements and displays them dynamically on the webpage.

2.6 Example of JSON Product Data

JSON stands for JavaScript Object Notation. It is a lightweight data format used to exchange information between applications.

Example:


```
[
  {
    "id": 101,
    "name": "Laptop",
    "price": 45000,
    "category": "Electronics",
    "availability": "In Stock"
  },

  {
    "id": 102,
    "name": "Laptop Bag",
    "price": 1200,
    "category": "Accessories",
    "availability": "In Stock"
  },
```

```
  {
    "id": 103,
    "name": "Mobile Phone",
    "price": 18000,
    "category": "Electronics",
    "availability": "In Stock"
  }
]
```


Explanation:

- id represents the product identification number.
- name represents the product name.
- price represents the product price.
- category represents the product category.
- availability indicates whether the product is available.

2.7 Algorithm

1. Start the program.
2. Create an HTML search webpage.
3. Add a search input field.
4. Create a product display section.
5. Capture the user's search keyword using JavaScript.
6. Send an asynchronous request to the server using AJAX.
7. Retrieve product details in JSON format.
8. Convert the JSON response into JavaScript objects.
9. Compare the product names with the search keyword.
10. Display the matching products using DOM manipulation.
11. Display a message if no products are found.
12. Stop the program.

2.8 Program

Save the following code as product.html.

This example assumes that a server endpoint named /api/products is available.

```
<!DOCTYPE html>
<html>
<head>
    <title>Dynamic Product Search</title>
</head>
<body>

<h2>E-Commerce Product Search</h2>

<input type="text" id="searchBox"
placeholder="Enter product name">

<p id="status"></p>

<div id="productList"></div>

<script>

let searchBox = document.getElementById("searchBox");

let productList = document.getElementById("productList");

let statusText = document.getElementById("status");

searchBox.addEventListener("input", function() {

    let keyword = searchBox.value.trim();
```

```
productList.innerHTML = "";

if (keyword === "") {
    statusText.textContent = "Enter a product name.";
    return;
}

statusText.textContent = "Searching...";

fetch("/api/products?q=" + encodeURIComponent(keyword))

.then(function(response) {

    if (!response.ok) {
        throw new Error("Server request failed");
    }

    return response.json();

})

.then(function(products) {

    productList.innerHTML = "";

    if (products.length === 0) {
        statusText.textContent = "No products found.";
        return;
    }
```

```
statusText.textContent =
products.length + " products found.";

products.forEach(function(product) {

    let card = document.createElement("div");

    let name = document.createElement("h3");

    name.textContent = product.name;

    let price = document.createElement("p");

    price.textContent = "Price: Rs. " + product.price;

    let category = document.createElement("p");

    category.textContent =
    "Category: " + product.category;

    let availability = document.createElement("p");

    availability.textContent =
    "Availability: " + product.availability;

    card.append(name, price, category, availability);

    productList.appendChild(card);
```

```
        });

    })

    .catch(function(error) {

        statusText.textContent =
        "Unable to load products. Please try again.";

        console.error(error);

    });

});

</script>

</body>
</html>
```

2.9 Server-Side Program

The following example demonstrates how the server can search products and return JSON data.

It uses Node.js and Express.

```
const express = require("express");

const app = express();

const products = [
```

```
{
  id: 101,
  name: "Laptop",
  price: 45000,
  category: "Electronics",
  availability: "In Stock"
},

{
  id: 102,
  name: "Laptop Bag",
  price: 1200,
  category: "Accessories",
  availability: "In Stock"
},

{
  id: 103,
  name: "Mobile Phone",
  price: 18000,
  category: "Electronics",
  availability: "In Stock"
},

{
  id: 104,
  name: "Wireless Mouse",
  price: 650,
```

```
        category: "Accessories",
        availability: "In Stock"
    }

];

app.get("/api/products", function(req, res) {

    let keyword = (req.query.q || "").toLowerCase();

    let result = products.filter(function(product) {

        return product.name.toLowerCase().includes(keyword);

    });

    res.json(result);

});

app.listen(3000, function() {

    console.log("Server running on port 3000");

});
```

Explanation:

1. Express creates the web server.
2. The product array stores sample product details.
3. The API receives the search keyword.

4. The filter() method identifies matching products.
5. The res.json() method sends the matching products as JSON.
6. The browser receives and displays the response.

2.10 Sample Output

E-COMMERCE PRODUCT SEARCH

Search: Lap

2 products found.

Product Name: Laptop

Price: Rs. 45000

Category: Electronics

Availability: In Stock

Product Name: Laptop Bag

Price: Rs. 1200

Category: Accessories

Availability: In Stock

2.11 Important JavaScript Methods Used

Method	Purpose
addEventListener()	Detects user input
fetch()	Sends an asynchronous request
response.json()	Converts response into JSON data
createElement()	Creates HTML elements

appendChild()	Adds elements to the webpage
forEach()	Processes each product
textContent	Displays product information
catch()	Handles request errors

2.12 Advantages

1. Provides fast product searching.
2. Displays matching products dynamically.
3. Avoids complete webpage reloading.
4. Uses JSON for data exchange.
5. Reduces unnecessary page navigation.
6. Improves the shopping experience.
7. Can be connected to a large product database.

2.13 Applications

- Online shopping websites.
- Electronic product stores.
- Grocery shopping applications.
- Clothing websites.
- Book-selling websites.
- Product inventory systems.

2.14 Conclusion

Thus, the dynamic e-commerce product search webpage uses JavaScript DOM manipulation and AJAX to retrieve product details from the server in JSON format. It displays matching products immediately when the customer enters a product name. This approach improves webpage performance, reduces unnecessary page reloads and provides a better shopping experience.

QUESTION 3

Discuss the Significance of the XMLHttpRequest Object, Its readyState Properties and Callback Methods. Illustrate the Working of AJAX Using a Suitable JavaScript Program.

10 Marks

3.1 Aim

To understand the XMLHttpRequest object, its importance, readyState properties and callback methods, and to demonstrate AJAX communication between a browser and a server using JavaScript.

3.2 Introduction to AJAX

AJAX stands for Asynchronous JavaScript and XML.

It is a web development technique used to communicate between a browser and a server without reloading the complete webpage.

In traditional web applications, when users request new information, the browser may reload the entire page.

AJAX solves this problem by allowing JavaScript to communicate with the server in the background.

The server processes the request and sends the response. JavaScript updates only the required part of the webpage.

AJAX commonly uses JSON for exchanging data in modern web applications, although XML and text can also be used.

3.3 Significance of XMLHttpRequest Object

XMLHttpRequest is a built-in JavaScript browser object used to send HTTP requests and receive responses from a server.

It is an important component of AJAX applications.

Importance of XMLHttpRequest:

1. It allows communication between the browser and server.
2. It supports asynchronous requests.
3. It retrieves information without reloading the complete webpage.
4. It supports HTTP methods such as GET and POST.
5. It can receive data in JSON, XML and text formats.
6. It provides information about the progress of requests.
7. It supports error handling.
8. It helps create interactive and dynamic webpages.

3.4 Important Properties of XMLHttpRequest

Property	Description
readyState	Indicates the current state of the request

status	Contains the HTTP response status code
responseText	Contains the response as text
responseXML	Contains the response as an XML document, when applicable
onreadystatechange	Executes a callback when readyState changes
onload	Executes when the request completes at the network level
onerror	Executes when a network error occurs

3.5 readyState Properties

The readyState property indicates the current progress of an XMLHttpRequest.

It contains five different states.

1. UNSENT – readyState 0

- The XMLHttpRequest object has been created.
- The open() method has not been called.

2. OPENED – readyState 1

- The open() method has been called.
- The request is ready to be configured and sent.

3. HEADERS_RECEIVED – readyState 2

- The send() method has been called.
- Response headers have been received from the server.

4. LOADING – readyState 3

- The browser is receiving the response data.
- Partial response data may be available.

5. DONE – readyState 4

- The request has completed.
- The response is ready for processing.

3.6 readyState Diagram

AJAX

home.cs.colorado.edu

3.7 Explanation of Each State

readyState	State	Explanation
0	UNSENT	Request object created
1	OPENED	open() method called
2	HEADERS_RECEIVED	Server response headers received
3	LOADING	Response data is loading
4	DONE	Request completed

3.8 Callback Methods

A callback is a function that executes when a particular event occurs.

XMLHttpRequest provides several callback event handlers.

1. onreadystatechange

This callback executes whenever the readyState value changes.

It is commonly used to check whether the request is completed.

Example:

```
xhr.onreadystatechange = function() {

   if (xhr.readyState === 4) {

      if (xhr.status === 200) {

         console.log(xhr.responseText);

      }

   }

};
```

2. onload

This callback executes when the request completes at the network level.

Example:

```
xhr.onload = function() {

    console.log("Request completed");

};
```

3. onerror

This callback executes when a network error occurs.

Example:

```
xhr.onerror = function() {

    console.log("Network error occurred");

};
```

4. onprogress

This callback executes while response data is being received.

Example:

```
xhr.onprogress = function() {

    console.log("Loading data...");

};
```

5. ontimeout

This callback executes when the request exceeds the configured timeout period.

Example:

```
xhr.ontimeout = function() {

    console.log("Request timed out");
```

};

3.9 Working Principle of AJAX

AJAX works through the following steps.

1. The user performs an action on the webpage.
2. JavaScript creates an XMLHttpRequest object.
3. The open() method specifies the HTTP method and server URL.
4. A callback function is assigned to handle the response.
5. The send() method sends the request.
6. The server processes the request.
7. The server returns the required data.
8. JavaScript receives the response.
9. The DOM updates the required section of the webpage.

3.10 AJAX Architecture Diagram

Leçon 11 : AJAX par l'exemple

users.polytech.unice.fr

3.11 Algorithm

1. Start the program.
2. Create an HTML webpage.
3. Add a button to load student information.
4. Create an XMLHttpRequest object.
5. Use the open() method to initialise a GET request.
6. Define the onreadystatechange callback.
7. Check whether readyState is equal to 4.
8. Check whether the HTTP status is 200.
9. Retrieve the response using responseText.
10. Convert the JSON response into a JavaScript object.
11. Display the student information using DOM manipulation.
12. Display an error message if the request fails.
13. Stop the program.

3.12 JavaScript AJAX Program

Save the following code as ajax.html.

```
<!DOCTYPE html>
<html>
<head>
   <title>AJAX Example</title>
</head>

<body>

<h2>AJAX Data Loading Example</h2>

<button onclick="loadData()">
Load Student Details
</button>

<p id="output">
Student information will appear here.
</p>

<script>

function loadData() {

   const xhr = new XMLHttpRequest();

   xhr.open("GET", "student.json", true);

   xhr.onreadystatechange = function() {
```

```
    if (xhr.readyState === 4) {

        if (xhr.status === 200) {

            const student = JSON.parse(xhr.responseText);

            document.getElementById("output").textContent =
            "Name: " + student.name +
            " | Register Number: " + student.registerNo +
            " | Department: " + student.department;

        }

        else {

            document.getElementById("output").textContent =
            "Unable to load student information.";

        }

    }

};

xhr.onerror = function() {

    document.getElementById("output").textContent =
    "Network error occurred.";
```

```
    };

    xhr.send();

}

</script>

</body>
</html>
```

3.13 JSON File

Create another file named student.json in the same server folder.

```
{
    "name": "Sai Venkata",
    "registerNo": "123456",
    "department": "CSE"
}
```

This JSON file contains the student information that the AJAX program retrieves.

3.14 Explanation of the Program

Step 1: Creating XMLHttpRequest

```
const xhr = new XMLHttpRequest();
```

This creates a new XMLHttpRequest object.

Step 2: Opening the request

```
xhr.open("GET", "student.json", true);
```

This prepares an asynchronous GET request to retrieve student.json.

Step 3: Checking readyState

```
if (xhr.readyState === 4)
```

This checks whether the request is completed.

Step 4: Checking HTTP status

if (xhr.status === 200)

This checks whether the server returned a successful HTTP response.

Step 5: Processing JSON

const student = JSON.parse(xhr.responseText);

This converts the received JSON text into a JavaScript object.

Step 6: Updating webpage

The textContent property displays the received student information on the webpage.

Step 7: Sending the request

xhr.send();

This sends the request to the server.

3.15 Sample Output

Before clicking the button:

AJAX Data Loading Example

[Load Student Details]

Student information will appear here.

After clicking the button:

AJAX Data Loading Example

[Load Student Details]

Name: Sai Venkata

Register Number: 123456

Department: CSE

3.16 Advantages of AJAX

1. Reduces complete webpage reloads.
2. Improves webpage responsiveness.

3. Allows background communication with the server.
4. Supports JSON and XML data exchange.
5. Improves user experience.
6. Reduces unnecessary page navigation.
7. Supports dynamic web applications.

3.17 Applications of AJAX

- Google search suggestions.
- Online shopping websites.
- Social media applications.
- Live notification systems.
- Chat applications.
- Online banking applications.
- Dynamic data-loading webpages.

3.18 Conclusion

Thus, XMLHttpRequest plays an important role in AJAX-based web applications. Its readyState properties indicate the progress of a request, and callback methods help process server responses and errors. The example demonstrates how student information can be retrieved from a server and displayed dynamically without reloading the complete webpage.

QUESTION 4

Develop a TypeScript Program Using a Class and Object to Represent Student Details. Include Properties, a Constructor and a Method to Display Student Information.

10 Marks

4.1 Aim

To develop a Student Management System using TypeScript. The program uses a class and object to manage student information, including name, register number, department and marks. It includes properties, a constructor and a method to display student information.

4.2 Introduction to TypeScript

TypeScript is a programming language developed by Microsoft.

It is a superset of JavaScript that provides additional features such as static typing, classes, interfaces and object-oriented programming.

TypeScript helps developers write structured and maintainable programs.

It identifies many type-related errors during development.

TypeScript code is compiled into JavaScript before execution.

It is useful for developing large web applications and software projects.

4.3 Important Features of TypeScript

1. Supports static typing.
2. Supports classes and objects.
3. Provides constructors.
4. Supports inheritance.
5. Supports interfaces.
6. Improves code readability.
7. Helps identify errors during compilation.
8. Supports object-oriented programming.

4.4 Object-Oriented Programming Concepts

1. Class

A class is a blueprint used to create objects.

In this program, Student is the class.

2. Object

An object is an instance of a class.

It stores actual values for the properties defined in the class.

3. Properties

Properties are variables declared inside a class.

They store the information of an object.

4. Constructor

A constructor is a special method that executes automatically when an object is created.

It is used to initialise the properties of the object.

5. Method

A method is a function defined inside a class.

It performs a particular operation.

In this program, displayDetails() displays student information.

4.5 Student Management System Architecture

Solved The UML Student class diagram is shown below. You are | Chegg.com

chegg.com

4.6 Class Structure

The Student class contains the following properties and methods.

Component	Description	Data type
name	Student name	string
registerNumber	Register number	string
department	Department name	string
marks	Student marks	number
constructor()	Initialises properties	Constructor
displayDetails()	Displays information	void

4.7 Algorithm

1. Start the program.
2. Create a class named Student.
3. Declare the name property with string type.
4. Declare the registerNumber property with string type.
5. Declare the department property with string type.
6. Declare the marks property with number type.
7. Create a constructor with four parameters.
8. Assign the constructor parameters to the class properties using the this keyword.
9. Create a method named displayDetails().
10. Display all student information using console.log().
11. Create an object of the Student class using the new keyword.
12. Call the displayDetails() method using the object.
13. Stop the program.

4.8 TypeScript Program

Save the program as Student.ts.

```
class Student {

    name: string;

    registerNumber: string;

    department: string;

    marks: number;

    constructor(
        name: string,
        registerNumber: string,
        department: string,
        marks: number
    ) {

        this.name = name;

        this.registerNumber = registerNumber;

        this.department = department;

        this.marks = marks;

    }

    displayDetails(): void {
```

```
        console.log("STUDENT INFORMATION");

        console.log("--------------------");

        console.log("Name: " + this.name);

        console.log("Register Number: " + this.registerNumber);

        console.log("Department: " + this.department);

        console.log("Marks: " + this.marks);

    }

}

// Creating an object

const student1 = new Student(
    "Sai Venkata",
    "123456",
    "CSE",
    87
);

// Calling the method

student1.displayDetails();
```

4.9 Explanation of the Program

Step 1: Creating a class

class Student

The class keyword creates a class named Student.

It acts as a blueprint for creating student objects.

Step 2: Declaring properties

name: string;

registerNumber: string;

department: string;

marks: number;

These properties store the student information.

The string data type is used for text values, and the number data type is used for marks.

Step 3: Creating the constructor

```
constructor(
    name: string,
    registerNumber: string,
    department: string,
    marks: number
)
```

The constructor receives four values when an object is created.

Step 4: Using the this keyword

this.name = name;

The this keyword refers to the current object.

It assigns the received value to the corresponding property.

Step 5: Creating a method

displayDetails(): void

This method displays the complete student information.

The void return type indicates that the method does not return a value.

Step 6: Creating an object

```
const student1 = new Student(
    "Sai Venkata",
    "123456",
    "CSE",
    87
);
```

The new keyword creates an object of the Student class.

The object stores the actual student details.

Step 7: Calling the method

```
student1.displayDetails();
```

This statement calls the displayDetails() method using the student1 object.

4.10 Sample Output

```
STUDENT INFORMATION
------------------

Name: Sai Venkata

Register Number: 123456

Department: CSE

Marks: 87
```

4.11 Creating Multiple Student Objects

The same class can be used to create multiple student objects.

Example:

```
const student2 = new Student(
    "Arun Kumar",
```

```
    "123457",
    "ECE",
    91
);

student2.displayDetails();
```

Output:

```
STUDENT INFORMATION
-------------------

Name: Arun Kumar

Register Number: 123457

Department: ECE

Marks: 91
```

This shows that one class can be reused to create multiple objects with different information.

4.12 Compilation and Execution

Follow these steps to execute the TypeScript program.

Step 1: Install Node.js.

Step 2: Install TypeScript using the terminal.

```
npm install -g typescript
```

Step 3: Save the program as:

Student.ts

Step 4: Compile the TypeScript program.

```
tsc Student.ts
```

Step 5: Execute the generated JavaScript file.

```
node Student.js
```

4.13 Advantages of TypeScript

1. Provides type safety.
2. Improves program readability.
3. Supports object-oriented programming.
4. Makes code easier to maintain.
5. Allows code reusability.
6. Helps identify errors during compilation.
7. Suitable for developing large applications.

4.14 Applications

- Student Management Systems.
- College administration applications.
- Employee Management Systems.
- Online examination systems.
- Library Management Systems.
- Educational web applications.

4.15 Conclusion

Thus, the Student Management System is successfully implemented using TypeScript classes and objects. The Student class contains properties, a constructor and a method to display student information. The object stores the actual student details and displays them using the method. This approach improves code organisation, reusability and maintainability.

QUESTION 5

Develop a JavaScript-Based Form Validation System for an Online Banking Application. Use Regular Expressions to Validate Account Number, Mobile Number and Email ID.

5.1 Aim

To develop a JavaScript-based online banking form validation system using Regular Expressions. The system validates the account number, mobile number and email ID and displays appropriate error messages for incorrect inputs.

5.2 Introduction

Online banking applications allow customers to access banking services through the internet.

Customers may use these applications for account registration, profile updates, banking information and service requests.

During online banking registration, customers must enter correct information such as account number, mobile number and email ID.

If incorrect information is entered, the application may not be able to identify or contact the customer correctly.

Therefore, form validation is an important part of online banking applications.

JavaScript is used to validate the information entered by customers. Regular Expressions are used to check whether the input follows the required format.

5.3 Technologies Used

HTML: Creates the online banking form.

CSS: Provides the layout and appearance of the form.

JavaScript: Performs the validation process.

DOM Manipulation: Reads input values and displays error messages.

Regular Expressions: Checks account number, mobile number and email ID formats.

5.4 Validation Requirements

The following validation conditions are used for the academic example.

Field	Validation rule
Account Number	Exactly 12 digits
Mobile Number	Exactly 10 digits and starts from 6 to 9

Email ID	Must contain a valid username, @ symbol and domain

Important: The 12-digit account-number condition is an example for this assignment. Actual bank account-number formats differ between banks and must be checked according to the relevant bank's specifications.

5.5 Regular Expression Patterns

Account Number

/^\d{12}$/

This pattern accepts exactly 12 digits.

Mobile Number

/^[6-9]\d{9}$/

This pattern accepts exactly 10 digits, beginning with 6, 7, 8 or 9.

Email ID

/^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$/

This pattern checks the basic structure of an email address.

5.6 Explanation of Regular Expression Symbols

Symbol	Meaning
^	Beginning of input
$	End of input
\d	Any digit from 0 to 9
{12}	Exactly 12 repetitions
{9}	Exactly 9 repetitions
[6-9]	Any digit between 6 and 9
+	One or more occurrences
.	Any character in regular expressions; escaped as \. when a literal dot is required

5.7 System Architecture Diagram

[알고리즘 디자인 설계 : Flowchart와 Psuedo-code]

[thsd-stjd.tistory.com]

Architecture explanation:

1. Customer opens the online banking form.
2. Customer enters account number, mobile number and email ID.
3. JavaScript retrieves the entered values.
4. Regular Expressions validate each input.
5. Incorrect inputs generate error messages.
6. Correct inputs generate a success message.
7. The validated information can then be processed by the application.

5.8 Algorithm

1. Start the program.
2. Create an online banking registration form.
3. Add input fields for account number, mobile number and email ID.
4. Create separate error-message areas.
5. Attach a submit event listener to the form.
6. Read the entered values using DOM methods.
7. Validate the account number using a Regular Expression.
8. Validate the mobile number using a Regular Expression.
9. Validate the email ID using a Regular Expression.
10. If any input is invalid, display an appropriate error message.
11. If all inputs are valid, display a success message.
12. Stop the program.

5.9 JavaScript Program

Save the program as banking.html.

```
<!DOCTYPE html>
<html>
<head>

   <title>Online Banking Validation</title>

</head>
```

```
<body>

<h2>Online Banking Customer Form</h2>

<form id="bankForm">

    Account Number:

    <input type="text" id="account">

    <p id="accountError"></p>


    Mobile Number:

    <input type="text" id="mobile">

    <p id="mobileError"></p>


    Email ID:

    <input type="text" id="email">

    <p id="emailError"></p>


    <button type="submit">Submit</button>
```

```
</form>

<h3 id="message"></h3>


<script>

document.getElementById("bankForm")
.addEventListener("submit", function(event) {

    event.preventDefault();

    let account =
    document.getElementById("account").value.trim();

    let mobile =
    document.getElementById("mobile").value.trim();

    let email =
    document.getElementById("email").value.trim();



    let accountPattern = /^\d{12}$/;

    let mobilePattern = /^[6-9]\d{9}$/;

    let emailPattern =
    /^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$/;
```

```
let valid = true;

document.getElementById("accountError").textContent = "";

document.getElementById("mobileError").textContent = "";

document.getElementById("emailError").textContent = "";

document.getElementById("message").textContent = "";



if (!accountPattern.test(account)) {

    document.getElementById("accountError").textContent =
    "Account number must contain exactly 12 digits.";

    valid = false;

}



if (!mobilePattern.test(mobile)) {

    document.getElementById("mobileError").textContent =
    "Enter a valid 10-digit mobile number.";

    valid = false;
```

```
    }

    if (!emailPattern.test(email)) {
        document.getElementById("emailError").textContent =
        "Enter a valid email ID.";
        valid = false;
    }

    if (valid) {
        document.getElementById("message").textContent =
        "All details are valid. Form submitted successfully!";
        document.getElementById("bankForm").reset();
    }
});
</script>

</body>
</html>
```

5.10 Explanation of the Program

Step 1: Creating the banking form

HTML creates input fields for account number, mobile number and email ID.

Step 2: Accessing input fields

The getElementById() method accesses the form elements using their IDs.

Step 3: Reading the entered values

The value property retrieves the values entered by the customer.

Step 4: Defining Regular Expressions

Three Regular Expressions are created to validate account number, mobile number and email ID.

Step 5: Validating account number

The account number is checked using the pattern /^\d{12}$/.

It accepts exactly 12 digits.

Step 6: Validating mobile number

The mobile number is checked using /^[6-9]\d{9}$/.

It accepts a 10-digit number beginning with 6 to 9.

Step 7: Validating email ID

The email address is checked using the email regular expression.

It checks for the basic email format.

Step 8: Displaying error messages

The textContent property displays error messages for invalid information.

Step 9: Displaying success message

If all the inputs are valid, the application displays a successful submission message.

5.11 Sample Output

Valid Input

ONLINE BANKING CUSTOMER FORM

Account Number: 123456789012

Mobile Number: 9876543210

Email ID: customer@gmail.com

[Submit]

All details are valid.

Form submitted successfully!

Invalid Input

ONLINE BANKING CUSTOMER FORM

Account Number: 12345678

Mobile Number: 12345

Email ID: customer.gmail.com

[Submit]

Error: Account number must contain exactly 12 digits.

Error: Enter a valid 10-digit mobile number.

Error: Enter a valid email ID.

5.12 Test Cases

Test case	Input	Expected result
1	123456789012	Valid account
2	12345678	Invalid account

3	12345678901A	Invalid account
4	9876543210	Valid mobile
5	5123456789	Invalid mobile
6	customer@gmail.com	Valid email
7	customer.gmail.com	Invalid email

5.13 Advantages

1. Reduces incorrect customer information.
2. Improves the accuracy of banking forms.
3. Provides immediate error messages.
4. Improves the customer experience.
5. Uses simple JavaScript methods.
6. Makes form validation easier.
7. Helps prevent common input-format mistakes.

5.14 Security Considerations

Online banking applications must provide strong security because they handle sensitive customer information.

The following security measures are important:

1. Use HTTPS to protect information during transmission.
2. Validate all information again on the server.
3. Use secure authentication methods.
4. Protect customer account information.
5. Restrict unauthorised access.
6. Never store passwords or PINs as plain text.
7. Verify account ownership through secure banking procedures.

Regular Expressions only check the format of an account number. They cannot confirm whether the account actually exists or belongs to the customer.

5.15 Applications

- Online banking registration.
- Customer profile management.

- Bank account service requests.
- Internet banking applications.
- Digital payment applications.
- Customer information systems.

5.16 Conclusion

Thus, the JavaScript-based online banking form validation system successfully validates account number, mobile number and email ID using Regular Expressions. It displays appropriate error messages when incorrect information is entered and allows successful submission when all inputs are valid. This improves the accuracy and usability of online banking applications.

OVERALL CONCLUSION

This assignment explains five important concepts of Internet Programming: JavaScript DOM manipulation, Regular Expressions, AJAX, XMLHttpRequest and TypeScript.

The first question demonstrates student registration form validation using JavaScript and Regular Expressions. It validates student name, email ID, mobile number and password without reloading the webpage.

The second question explains dynamic e-commerce product searching using JavaScript DOM manipulation and AJAX. It retrieves product information from a server in JSON format and displays matching products dynamically.

The third question discusses the importance of XMLHttpRequest, its five readyState properties and callback methods. It also demonstrates AJAX communication between the browser and server.

The fourth question demonstrates object-oriented programming using TypeScript. It creates a Student class with properties, a constructor, an object and a method to display student information.

The fifth question demonstrates an online banking form validation system using JavaScript and Regular Expressions to validate account number, mobile number and email ID.

Overall, these technologies are useful for developing interactive, dynamic, reliable and user-friendly web applications.